Dgrammar and DPGrammar: Frontier Constrained Decoding for Diffusion Language Models

Jeng-Yue Liu[1], Wilson Zheng[1], Haoling Pu[1]
[1]Language Technologies Institute, Carnegie Mellon University
{buffettl,wilsonz,haolingp}@andrew.cmu.edu

Abstract

The leading constrained decoding method for discrete diffusion language models (dLLMs), LAVE (Zhang et al., 2026), reports ≈99% syntactic validity on simple JSON benchmarks. We show this optimism does not transfer: on JSONSchemaBench (Geng et al., 2025) `medium`—a rigorous benchmark with nested schemas, cross-field ordering constraints, and mixed-type arrays—our reimplementation of LAVE achieves only **68.8%** schema validity (n=586), consuming 222.6 lookahead draws per instance and timing out frequently. We trace the failure to LAVE's stochastic suffix sampling: random completions rarely cover the joint constraint space of hard schemas. We address this in two steps. **Dgrammar** is a deterministic engineering redesign: incremental LLGUIDANCE (Microsoft, 2025) frontier masking, AIMD adaptive batching, in-place violator resampling, and async `compute_mask` overlap—no random completions drawn. It raises validity to **84.9%** and reduces median time by 2.8×. **DPGrammar** adds a Viterbi-style dynamic program (`dp_fix_prefix`) that jointly reassigns violated spans under one forward's log probabilities over grammar DFA states, pushing validity to **99.2%** (mean resamples: 1.06). Our results show that LAVE's high reported validity is benchmark-specific, and that deterministic frontier parsing plus principled segment repair are sufficient to recover—and exceed—it on harder schemas.

1 Introduction

Many applications require model outputs to satisfy a formal grammar or schema: APIs that consume JSON, tools that compile code, pipelines that validate structured records. Discrete diffusion language models (dLLMs) such as LLaDA (Nie et al., 2025) are attractive because they refine many tokens simultaneously under bidirectional context, but this parallel update breaks the monotonic prefix on which classical Finite-State Machine (FSM)[1] constrained decoding is built. Ensuring schema compliance without destroying the model's joint proposals is therefore an open problem with direct deployment consequences.

The leading constrained decoding method for dLLMs is LAVE (Zhang et al., 2026). While LAVE achieves near-perfect validity on `json-mode-eval` (Nous Research, 2024), our evaluation on the more complex JSONSchemaBench (Geng et al., 2025) `medium` reveals a performance gap, with validity decreasing to 68.8% due to the benchmark's intricate nested structures and cross-field constraints. The gap is not incidental: LAVE verifies each frontier token by drawing K random completions and accepting only if at least one is grammar-valid. On simple schemas, random suffixes are likely to be valid; on schemas with joint multi-token constraints, the probability that a random suffix satisfies all constraints simultaneously drops sharply, causing false rejections, K-fold forward cost per token, and stochastic run-to-run instability.

We address the root cause in two steps. **Dgrammar** replaces stochastic sampling with deterministic frontier masking: grammar constraints use the incremental `LLMatcher` from LLGUIDANCE (Microsoft, 2025) at the first unresolved position only; committed spans advance the parser state without replaying from scratch; AIMD[2] adaptive batching unmasks a block per forward pass; violators are localized and resampled in-place; and `compute_mask` runs asynchronously overlapping the GPU forward, where no random completions drawn. This engineering redesign raises validity to 84.9% and cuts median time by 2.8×.

[1]FSM refers to the automaton that tracks which tokens are valid at each position given the grammar/schema.

[2]Adapted from the Additive Increase/Multiplicative Decrease (AIMD) principle in TCP congestion control: B increases by 1 upon success and resets to 1 upon a grammar violation.

When a batch unmask produces tokens with interdependent joint violations (e.g., a key committed in the wrong order forces its value into an incompatible type slot), greedy per-position repair still falls short. **DPGrammar** adds `dp_fix_prefix`: a Viterbi dynamic program that jointly reassigns the entire violated span $[c, s)$ over grammar DFA states and top-K token candidates in one pass, pushing validity to **99.2%** with mean resamples of 1.06.

Our contributions are: (i) identifying and quantifying LAVE's benchmark gap, tracing it to stochastic sampling under complex joint constraints; (ii) a deterministic engineering redesign achieving 84.9% validity; (iii) a principled Viterbi repair layer achieving 99.2%, matching LAVE's claimed number, now on a harder benchmark; and (iv) fine-grained timing instrumentation separating grammar-check from forward-pass cost.

2 Related Work

Discrete diffusion and parallel decoding. dLLMs such as LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) enable non-autoregressive, any-order text generation through iterative masked refinement. This flexibility introduces the *incomplete prefix problem*: unlike AR models, where a stable left-to-right prefix always exists for grammar-state tracking, dLLMs must predict tokens surrounded by still-masked or partially denoised positions, making it unclear where the parser frontier lies. Blockwise accelerators (e.g., DFlash (Chen et al., 2026), Spiffy (Agrawal et al., 2026)) reduce sampling steps but remain unconstrained, offering no structural or syntactic guarantees. Our work is orthogonal: we fix the backbone and study how to impose grammar constraints despite the incomplete prefix.

Constrained decoding in autoregressive models. Tooling such as Outlines (Willard and Louf, 2023) and XGrammar (Dong et al., 2024) exploits a monotonic prefix to maintain FSM or parser states across generation steps. These interfaces do not transfer to diffusion because there is no single causal frontier until we define one.

Constrained decoding for diffusion. Mündler et al. (2025) propose the first CFG-constrained dLLM decoder; DINGO (Suresh et al., 2025) uses dynamic programming for syntactic correctness at higher per-step cost. LAVE (Zhang et al., 2026) introduces a lookahead-then-verify mechanism—sampling K random completions to decide whether to accept each frontier token—and has become the dominant baseline, reporting $\approx$99% syntactic validity on json-mode-eval. We show this number is benchmark-specific: LAVE's stochastic sampling fails to cover joint constraint spaces on harder schemas, dropping to 68.8% on JSONSchemaBench medium, and its reliance on random completions risks false rejection of genuinely valid tokens. Plan-style decoders (e.g., PVF (Li et al., 2026)) emphasize structure-aware planning; we stay closer to token-level masking with an explicit incremental matcher.

Sparse transition structures and DP cost. STATIC (Su et al., 2026) shows that sparse DFA transition structures can be flattened into Compressed Sparse Row (CSR) matrices, enabling $O(K)$ coalesced lookups per step—where K is the grammar branch factor—rather than scanning the full vocabulary $|V|$. We adapt this principle to DPGrammar: instead of scoring all $|V|$ tokens at each DP position, we restrict the Viterbi search to top-K candidates per position, reducing per-step transition cost from $O(|V|)$ to $O(K)$ while preserving coverage of the most probable grammar-feasible paths.

Parallel verification and joint decisions. Speculative decoding (Leviathan et al., 2023) motivates verifying multiple tokens under one forward; LAVE relates in spirit but uses random suffixes. Dgrammar instead commits through a single incremental parser trail; DPGrammar adds a joint Viterbi layer over the violated span when that trail conflicts after a batch unmask.

3 Data

We evaluate on `jsb_medium`, a JSON Schema benchmark split used in our DLLM evaluation harness, aligned with the JSON schema structured output setting discussed in broader benchmarks (Geng et al., 2025). Each instance provides a natural language style prompt, a schema string, and evaluation extracts structured output from the model generation region.

Scale and splits. We report merge protocols aligned with Table 1. **LAVE** uses a nine shard Modal run covering n=586 instances (seed 0, T=128). **Dgrammar** and **DPGrammar** use nine paired shards with offsets $0, 66, \ldots, 528$, JSONL logs merged by `instance_id`, yielding n=511

unique instances per method. LAVE and Dgrammar/DPGrammar therefore differ in which instances appear in each column; the controlled comparison is between Dgrammar and DPGrammar on the same 511 IDs.

Preprocessing. Instances without a usable schema string are skipped in the driver. No additional manual annotation is performed; validity is judged by the benchmark's extraction and schema check, surfaced as the boolean `valid` in result JSON lines.

4 Method

4.1 Baselines reproduced

Our primary external baseline is **LAVE** (Zhang et al., 2026), reimplemented to operate with our LLaDA 8B Instruct backbone and `jsb_medium` loader under the same diffusion step budget T=128 where applicable. This isolates the constraint algorithm rather than changing the base model. We additionally compare **Dgrammar** (async greedy, `v2_async_ac4_timed`) against **DPGrammar** (`generate_dp`) on identical instances.

4.2 Dgrammar: proposed deterministic frontier stack

We implement schema constraints with LLGUIDANCE (Microsoft, 2025) and its `LLMatcher`: incremental `try_consume_tokens` reuses state instead of reparsing from scratch.

Constraint at the frontier only. `compute_mask(vocab_size)` is applied at the first unresolved position. Other positions may remain masked; we consume left to right over revealed runs. This avoids globally masking every row before the block is coherent.

Incremental prefix semantics. Committed non mask spans advance the matcher across denoising iterations without replaying from the prompt on every check.

Adaptive commit (AIMD style). After successful consumes we increase batch size up to $k_{\max}$=8; collisions reset width and identify the first violator.

Violator remask and in place resampling. We remask the violator, zero its logit in the current forward tensor, and iterate alternative argmax choices without always launching a new GPU forward.

Early accept and async overlap. We stop when `is_accepting()` holds. `compute_mask` for the next frontier runs in a CPU thread overlapping the upcoming forward.

4.3 DPGrammar: segment DP on top of Dgrammar

DPGrammar uses the *same* loop as Dgrammar; the only change is repair at violations. When consumption fails at c, we optimize the span $[c, s)$ up to the next mask with `dp_fix_prefix`: Viterbi style DP with DFA keys from `matcher.compute_logit_bias()`, top K=100 tokens per position in our runs, merging states and backtracking token replacements. If no feasible path exists, we fall back to Dgrammar's greedy remask. Figure 1 illustrates the shared stack and the DP attachment.

4.4 Completion after generation

Both systems share driver side repair after the diffusion phase: grammar guided greedy autocomplete, masked extension, grammar only closing, and minimal JSON synthesis from the schema when required.

4.5 Implementation and compute

Code builds on PyTorch, Transformers, LLGUIDANCE (Microsoft, 2025), and the constrained diffusion CD4d style bindings; benchmarks run on **Modal** with NVIDIA **A100** GPUs in our experiments. Backbone is **GSAI ML LLaDA 8B Instruct**. Block length 32 for timed async runs, remasking `low_confidence`, temperature 0.2 in the driver, seed 0 unless noted. Full hyperparameter tables can be folded into an appendix if space is tight.

4.6 Experimental design and metrics

Schema validity is the primary outcome (binary `valid` per instance). **Wall time** per instance measures end to end driver time. We report median, mean, and P95 to capture heavy tails (LAVE hits a 120 s cap in our run; DPGrammar shows rare long runs). **Effective constraint %** aggregates grammar check time, token selection, mask wait, and autocomplete mask time relative to forward plus overhead, as emitted by `run_dgrammar_timed.py`. **Resamples** counts remasking events. We did not compute formal confidence intervals; differences are large on validity, but future work should add bootstrap CIs on the same split.

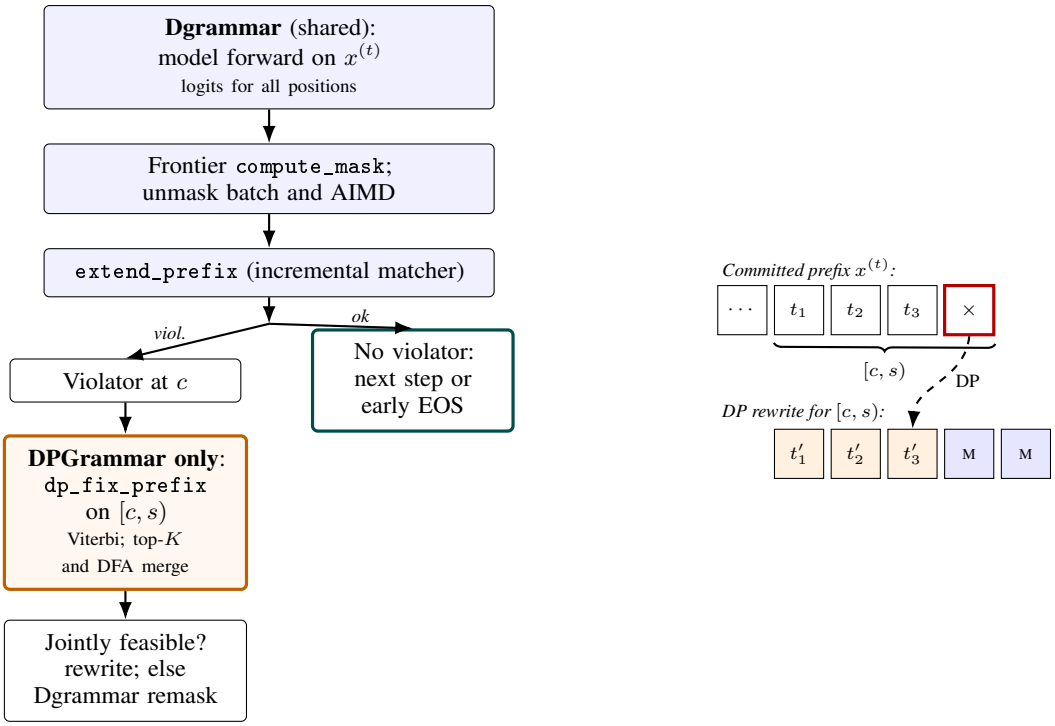

Figure 1: **Dgrammar** (blue tinted): one denoising step applies a full model forward, frontier masking and AIMD unmasking, and incremental `extend_prefix`. **DPGrammar** (orange): when a violator is detected at position c, `dp_fix_prefix` jointly rewrites the span $[c, s)$ via Viterbi over top-K tokens before falling back to greedy remasking.

5 Results

Table 1 reports aggregate metrics. We discuss results in the two-step framing of our approach.

LAVE vs. Dgrammar (engineering step). LAVE reaches 68.8% validity (n=586) with median wall time 28.45 s, P95 78.32 s, and 222.6 mean lookahead draws—many instances hit the 120 s timeout. Dgrammar replaces stochastic sampling with deterministic frontier masking: validity rises to 84.9% (n=511), median time drops to 10.23 s, and mean resamples fall to 32.43. The engineering redesign—incremental parser state, AIMD batching, async overlap—accounts for the entire improvement; no new theoretical primitive is needed for this gain.

Dgrammar vs. DPGrammar (principled step). Dgrammar's residual failures stem from greedy per-position repair that misses joint constraint structure (e.g., fixing a key but leaving an incompatible value type in the same span). DPGrammar adds `dp_fix_prefix`: a Viterbi search over the full violated span under grammar DFA states. Validity rises from 84.9% to 99.2% on the same 511 instances, mean resamples collapse from 32.43 to 1.06, with only a modest median latency increase (10.23 s → 12.87 s). The principled extension closes what the engineering step cannot: multi-token joint violations that greedy remasking iterates through slowly.

Qualitative observations. Error modes for the remaining 0.8% DPGrammar failures include completion-phase limits and schemas where minimal-JSON fallback is brittle. Detailed case studies are left to appendix space.

‡ LAVE nine-shard evaluation (n=586); median and P95 reflect sampling based verification cost.

§ LAVE resamples count lookahead verification draws, not token re-generations; the concept differs from Dgrammar/DPGrammar resamples.

6 Discussion

Interpretation. The LAVE–Dgrammar gap shows that deterministic frontier parsing is a strong engineering solution: it eliminates sampling variance and timeout risk with no new theoretical machinery. The Dgrammar–DPGrammar gap shows that greedy token-by-token repair has a structural limit—joint violations spanning multiple positions require a principled combinatorial

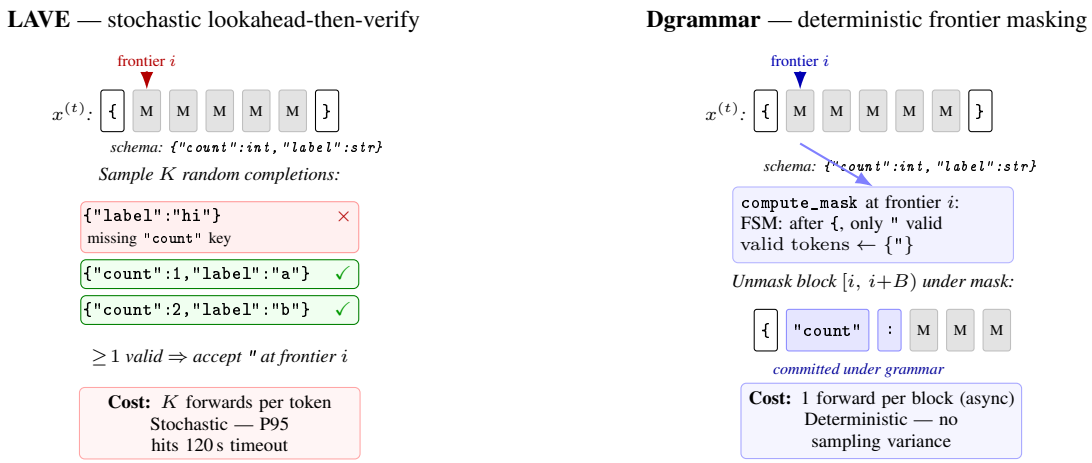

Figure 2: One denoising step under schema `{"count":int,"label":str}`. **LAVE** (Zhang et al., 2026) (left) samples K random completions at each frontier token, accepting it only when ≥ 1 suffix is valid—stochastic and $K\times$ more expensive per token, hitting the 120 s per-instance timeout on hard schemas. **Dgrammar** (right) runs `compute_mask` on the incremental FSM state and unmasks an entire block under that mask in one forward pass, overlapped with the next mask computation; the result is deterministic.

Method	Validity	Med. time (s)	Mean time (s)	P95 time (s)	Med. eff. const. %	Mean resamples
LAVE ‡ (reimpl., n=586)	68.8%	28.45	34.59	78.32	—	222.6 §
Dgrammar (async, n=511)	84.9%	10.23	13.14	32.10	20.43	32.43
DPGrammar (`dp`, n=511)	**99.2%**	12.87	14.96	28.55	**1.22**	**1.06**

Table 1: `jsb_medium` results (seed 0, T=128 steps). Dgrammar and DPGrammar are evaluated on the same 511 instances (nine-shard protocol); DPGrammar adds `dp_fix_prefix` on top. LAVE (Zhang et al., 2026) uses a nine-shard split covering 586 instances.

search, not more engineering tuning. The two steps are complementary: the engineering step is prerequisite infrastructure; the principled step addresses what remains.

Strengths. DPGrammar achieves the highest validity and lowest resample churn; Dgrammar offers a strong, practical system for settings where DP overhead is undesirable.

Weaknesses and surprises. Effective constraint % definitions differ in interpretability across methods; DPGrammar's median is low partly because fewer retries occur. Mean and median wall times are close for DPGrammar in our run. DPGrammar validity is below 100%, so completion and fallback stages still matter.

Implications. Practitioners should report forward versus constraint splits, not only total time. For research, joint repair on violation segments is a lightweight add on relative to full sequence DP (Suresh et al., 2025).

7 Conclusion and Future Work

We studied grammar constrained decoding for dLLMs in two steps. **Dgrammar** is an engineering redesign of the constraint loop—deterministic frontier masking, incremental parser state, AIMD batching, async overlap—that raises validity from 68.8% (LAVE) to 84.9% without any new theoretical primitive. **DPGrammar** is the principled extension: Viterbi-based joint repair over violated spans pushes validity to 99.2% and collapses resamples from 32.43 to 1.06, addressing multi-token joint constraints that greedy repair cannot resolve efficiently. **Broader takeaway:** the engineering step provides the infrastructure; the principled step provides the correctness guarantee—both are needed for near-perfect structured generation under diffusion.

Future work includes evaluating LAVE on the same nine shard split for a strictly controlled three way comparison; extending to additional JSON splits or code grammars; bootstrap confidence intervals; and exploring tighter integration with sparse automata acceleration (Su et al., 2026) for very large vocabularies.

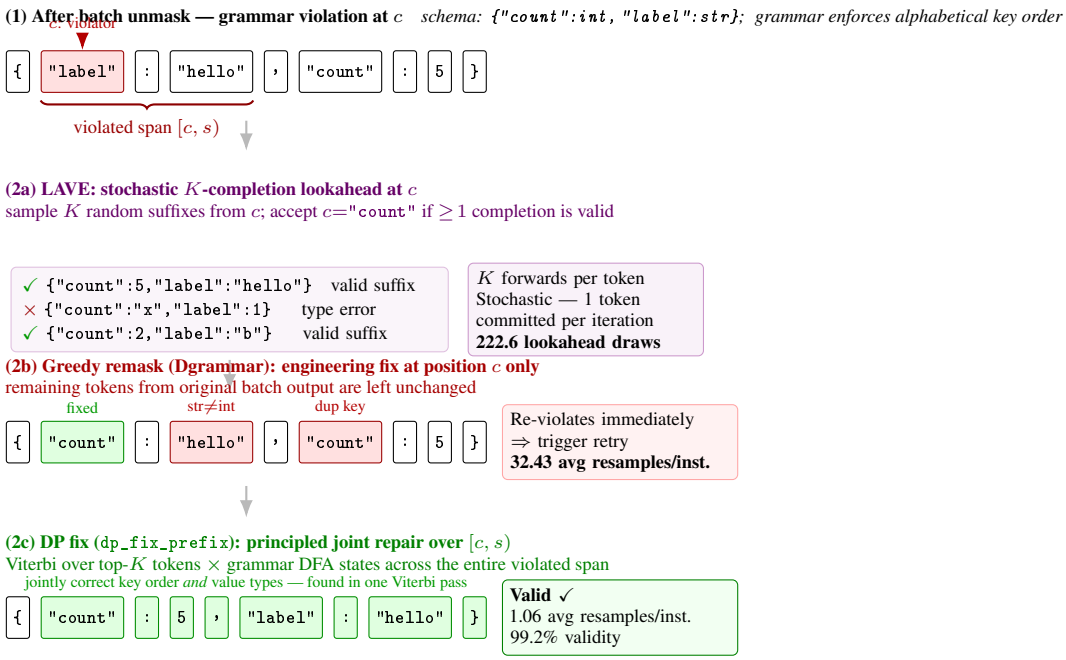

Figure 3: All three methods on the same violation. **(1)** A batch unmask produces `"label"` before `"count"`, violating the alphabetical key-order constraint; the violator at c spans $[c, s)$. **(2a) LAVE** samples K random suffixes from c and accepts the token only when ≥ 1 is valid—stochastic, commits one token per iteration, and accumulates 222.6 lookahead draws per instance on average. **(2b) Dgrammar**'s greedy remask is an engineering fix: the grammar mask forces `"count"` at c, but the value slot still holds `"hello"` (string in an integer field) and the original `"count"` key remains as a duplicate—causing immediate re-violation at 32.43 avg resamples per instance. **(2c) DPGrammar**'s `dp_fix_prefix` is the principled extension: Viterbi over top-K candidates across every position in $[c, s)$ simultaneously finds `"count":5,"label":"hello"`—correct order, correct types, no duplicates—in one pass, reducing to 1.06 avg resamples and 99.2% validity.

8 Limitations

LAVE and our methods are not evaluated on identical instance sets in Table 1, so cross column comparisons involving LAVE are indicative rather than strictly controlled. The validity flag depends on benchmark extraction; functional tests beyond schema syntax are not central here. Hyperparameter tuning was limited; different T, block sizes, or temperature could shift outcomes. DPGrammar explores top K tokens per DP position, which can miss globally optimal tokens outside the top K. Compute was cloud A100; other hardware changes overlap benefits.

9 Ethical Considerations

Structured generation can reduce malformed outputs in downstream automation, but *valid JSON is not always semantically safe or fair*: schemas can encode biased or incomplete coverage of user intent. Models and grammars should be reviewed for misuse in high stakes settings (medical, financial). Our benchmark instances originate from public style schema tasks; they may not represent all languages or cultures. Researchers and deployers should combine syntactic guarantees with content moderation and policy controls. We use open backbone weights and public style evaluation data; users of our code should respect model licenses and avoid deploying unreviewed systems in sensitive domains without human oversight.

References

Sudhanshu Agrawal, Risheek Garrepalli, Raghavv Goel, Mingu Lee, Christopher Lott, and Fatih Porikli. 2026. Spiffy: Multiplying diffusion llm acceleration via lossless speculative decoding. *Preprint*, arXiv:2509.18085.

Jian Chen, Yesheng Liang, and Zhijian Liu. 2026. Dflash: Block diffusion for flash speculative decoding. *Preprint*, arXiv:2602.06036.

Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems 7*.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel

Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. Jsonschemabench: A rigorous benchmark of structured outputs for language models. *Preprint*, arXiv:2501.10868.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. *Preprint*, arXiv:2211.17192.

Miao Li, Hanyang Jiang, Sikai Cheng, Hengyu Fu, Yuhang Cai, Baihe Huang, Tinghan Ye, Xuanzhou Chen, and Pascal Van Hentenryck. 2026. Plan, verify and fill: A structured parallel decoding approach for diffusion language models. *Preprint*, arXiv:2601.12247.

Microsoft. 2025. LLGuidance: Fast structured outputs for LLMs. GitHub repository.

Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2025. Constrained decoding of diffusion llms with context-free grammars. *Preprint*, arXiv:2508.10111.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large language diffusion models. *Preprint*, arXiv:2502.09992.

Nous Research. 2024. json-mode-eval. Hugging Face Datasets.

Zhengyang Su, Isay Katsman, Yueqi Wang, Ruining He, Lukasz Heldt, Raghunandan Keshavan, Shao-Chuan Wang, Xinyang Yi, Mingyan Gao, Onkar Dalal, Lichan Hong, Ed Chi, and Ningren Han. 2026. Vectorizing the trie: Efficient constrained decoding for llm-based generative retrieval on accelerators. *Preprint*, arXiv:2602.22647.

Tarun Suresh, Debangshu Banerjee, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. Dingo: Constrained inference for diffusion llms. *Preprint*, arXiv:2505.23061.

Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7b: Diffusion large language models. *Preprint*, arXiv:2508.15487.

Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. 2026. Lookahead-then-verify: Reliable constrained decoding for diffusion llms under context-free grammars. *Preprint*, arXiv:2602.00612.